CANAIS E MEMÓRIA COMPARTILHADA

FPPD — Unidade 3

Aulas 8, 9 e 11 — Março/Abril 2026

ROTEIRO

3 **Canais de Comunicação**
Conceitos: síncronos, assíncronos, bufferização

4 **Channels em Go**
make, envio/recepção, direcionais, close, range

5 **Memória Compartilhada em Go**
Mutex, RWMutex, detector de corrida, channels vs. mutexes

6 **Modelo Reativo e Select**
select, timeouts, topologias, fan-out/fan-in

CANAIS DE COMUNICAÇÃO

Conceitos fundamentais

O QUE É UM CANAL?

Um **canal** (channel) conecta um processo emissor a um processo receptor, permitindo troca de mensagens.

- Canais são **tipados**: o tipo da mensagem é declarado antes do uso
- Duas operações fundamentais: **enviar** e **receber**
- As operações podem ser **bloqueantes** — veremos como
- Em Go: `ch <- x` (enviar) e `y := <-ch` (receber)

CANAIS EM SISTEMAS OPERACIONAIS — PIPES

Em sistemas operacionais, canais são chamados de **pipes** — conectam a saída de um processo à entrada de outro.

```
ls -l *.txt | grep May | sort -k5 -n
```

- O operador `|` cria um canal entre dois processos
- A saída de `ls` flui para a entrada de `grep`, que flui para `sort`
- Cada processo executa **concorrentemente** — pipeline de dados
- Modelo poderoso: componentes simples conectados por canais

Pipes em Unix (1973) são um dos primeiros exemplos de comunicação por canais — Go adota a mesma filosofia com goroutines e channels.

CANAIS SÍNCRONOS (NÃO-BUFFERIZADOS)

Em um canal síncrono, emissor e receptor devem estar prontos **ao mesmo tempo** — rendez-vous.

EMISSOR

- Envia um valor para o canal
- **Bloqueia** até que alguém leia
- Só retoma quando o receptor consome o valor

RECEPTOR

- Tenta ler do canal
- **Bloqueia** até que alguém escreva
- Só retoma quando o emissor envia um valor

A sincronização é **implícita**: a comunicação só ocorre quando ambos estão prontos. Se nenhum parceiro aparecer, o processo fica bloqueado indefinidamente.

CANAIS ASSÍNCRONOS (BUFFERIZADOS)

Em um canal assíncrono, existe um **buffer** entre emissor e receptor — desacoplamento temporal.

EMISSOR

- Envia para o buffer
- **Não bloqueia** enquanto há espaço
- Bloqueia apenas quando o buffer está **cheio**

RECEPTOR

- Lê do buffer
- **Não bloqueia** enquanto há dados
- Bloqueia apenas quando o buffer está **vazio**

Desacoplamento temporal: emissor e receptor **não precisam estar prontos ao mesmo tempo**. Isso aumenta a concorrência, mas reduz a garantia de sincronização.

ESPECTRO DE BUFFERIZAÇÃO

A bufferização define o grau de acoplamento entre emissor e receptor.

Null	Single	Bounded	Unbounded
Sem buffer	1 posição	N posições	Ilimitado
Só síncrono	Cheio ou vazio	Caso mais comum	Não existe na prática

RELAÇÃO COM SINCRONISMO

- **Null/Single** → comunicação síncrona
- **Bounded** → comunicação assíncrona (até encher)
- **Unbounded** → sempre assíncrona (teórico)

RELAÇÃO COM GO

- `make(chan T)` → null buffer
- `make(chan T, 1)` → single
- `make(chan T, n)` → bounded

PROPRIEDADES DOS CANAIS

ENDEREÇAMENTO

- **Simétrico:** emissor e receptor se identificam mutuamente
- **Assimétrico:** emissor identifica o receptor, mas o receptor aceita de qualquer um
- Em Go, canais são objetos — quem tem a referência pode usar

DIREÇÃO

- **Bidirecional:** qualquer participante pode enviar ou receber
- **Unidirecional:** um lado só envia, outro só recebe
- Em Go: `chan<- T` (envio) e `<-chan T` (recepção)

Canais unidirecionais aumentam a **segurança**: restringem as operações que cada goroutine pode fazer, prevenindo erros de programação.

CHANNELS EM GO

Implementação prática

CRIANDO CHANNELS

Channels são criados com `make` e são **tipados** — cada canal carrega um tipo específico de dado.

```
ch := make(chan int)          // síncrono (sem buffer)
buf := make(chan string, 5) // assíncrono (buffer de 5)
```

ENVIO

```
ch <- 42        // envia para o canal
buf <- "hello" // envia para o buffer
```

RECEPÇÃO

```
v := <-ch       // recebe do canal
msg := <-buf    // recebe do buffer
```

O valor zero de um channel é `nil`. Enviar ou receber de um canal nil bloqueia **para sempre**.

CHANNELS SEM BUFFER

Canal síncrono — rendez-vous entre goroutines

```go
func main() {
    ch := make(chan int)

    go func() {
        fmt.Println("Enviando...")
        ch <- 42                // bloqueia até alguém receber
        fmt.Println("Enviado!")
    }()

    fmt.Println("Esperando...")
    valor := <-ch               // bloqueia até alguém enviar
    fmt.Println("Recebido:", valor)
}
```

O envio e a recepção são **pontos de sincronização**: a goroutine que chega primeiro espera pela outra. Garantia de que o dado foi entregue.

CHANNELS COM BUFFER

Canal assíncrono — desacoplamento temporal

```go
func main() {
    ch := make(chan int, 3)   // buffer com 3 posições

    ch <- 10                  // não bloqueia (buffer tem espaço)
    ch <- 20                  // não bloqueia
    ch <- 30                  // não bloqueia
    // ch <- 40              // bloquearia — buffer cheio!

    fmt.Println(<-ch)         // 10 (FIFO)
    fmt.Println(<-ch)         // 20
    fmt.Println(<-ch)         // 30
}
```

BLOQUEIA QUANDO...

- Envio com buffer **cheio**
- Recepção com buffer **vazio**

NÃO BLOQUEIA QUANDO...

- Envio com **espaço** no buffer
- Recepção com **dados** no buffer

CHANNELS DIRECIONAIS

Channels podem ser restritos a **somente envio** ou **somente recepção**, aumentando a segurança do código.

```go
func produtor(ch chan<- int) {   // ch: somente envio
    for i := 0; i < 10; i++ {
        ch <- i
    }
    close(ch)
}

func consumidor(ch <-chan int) { // ch: somente recepção
    for v := range ch {
        fmt.Println("Recebido:", v)
    }
}

func main() {
    ch := make(chan int, 5)      // bidirecional
    go produtor(ch)              // conversão implícita para chan<-
    consumidor(ch)               // conversão implícita para <-chan
}
```

O compilador **impede** operações inválidas: tentar receber de um `chan<-` ou enviar para um `<-chan` é um **erro de compilação**.

FECHAMENTO E ITERAÇÃO

CLOSE(CH)

- Sinaliza que **nenhum valor** mais será enviado
- Receptores **não bloqueiam** após o fechamento
- Receber de canal fechado retorna o **valor zero** do tipo
- Enviar para canal fechado causa **panic**

RANGE CH

- Itera sobre valores do canal
- Bloqueia esperando o próximo valor
- **Termina automaticamente** quando o canal é fechado
- Idiomático para consumir todos os valores

```go
ch := make(chan int, 5)
go func() {
    for i := 0; i < 5; i++ { ch <- i * i }
    close(ch)  // sinaliza fim
}()
for v := range ch {  // termina quando ch é fechado
    fmt.Println(v)   // 0, 1, 4, 9, 16
}
```

EXEMPLO PRÁTICO — CHANNELS EM AÇÃO

Combina `chan<-`, `<-chan`, `close`, `range` e `WaitGroup`

```go
func produtor(id int, ch chan<- string, wg *sync.WaitGroup) {
    for i := 0; i < 3; i++ {
        item := fmt.Sprintf("P%d-item%d", id, i)
        ch <- item
        fmt.Printf("[P%d] Produziu: %s\n", id, item)
    }
    wg.Done()
}

func consumidor(id int, ch <-chan string) {
    for item := range ch {
        fmt.Printf("[C%d] Consumiu: %s\n", id, item)
    }
}

func main() {
    ch := make(chan string, 5)
    var wg sync.WaitGroup
    for i := 1; i <= 3; i++ { wg.Add(1); go produtor(i, ch, &wg) }
    for i := 1; i <= 2; i++ { go consumidor(i, ch) }
    wg.Wait()
    close(ch)
}
```

MEMÓRIA COMPARTILHADA EM GO

Mutex e controle de acesso

CONDIÇÃO DE CORRIDA

Quando múltiplas goroutines acessam dados compartilhados **sem proteção**, o resultado depende da ordem de execução — uma **data race**.

```go
var contador int

func incrementar(wg *sync.WaitGroup) {
    for i := 0; i < 1000; i++ {
        contador++   // NÃO é atômico: ler → incrementar → escrever
    }
    wg.Done()
}

func main() {
    var wg sync.WaitGroup
    wg.Add(2)
    go incrementar(&wg)
    go incrementar(&wg)
    wg.Wait()
    fmt.Println(contador)   // ≠ 2000 na maioria das execuções!
}
```

O PROBLEMA DA SEÇÃO CRÍTICA

Proteger o acesso a dados compartilhados exige um protocolo que garanta **três propriedades**.

EXCLUSÃO MÚTUA

Apenas **um processo** por vez pode estar na seção crítica

PROGRESSO

Se ninguém está na seção crítica, a decisão de quem entra **não pode ser adiada indefinidamente**

ESPERA LIMITADA

Existe um **limite** no número de vezes que outros entram antes de um processo que está esperando

Soluções por software puro são complexas e ineficientes. Na prática, usamos primitivas de hardware e de linguagem: **mutexes**.

SYNC.MUTEX

`sync.Mutex` implementa exclusão mútua: apenas uma goroutine pode segurar o lock por vez.

```go
var (
    contador int
    mu       sync.Mutex
)

func incrementar(wg *sync.WaitGroup) {
    wg.Done()
    for i := 0; i < 1000; i++ {
        mu.Lock()       // entra na seção crítica
        contador++      // acesso exclusivo
        mu.Unlock()     // sai da seção crítica
    }
}
```

- `Lock()` — adquire o lock; bloqueia se já está travado
- `Unlock()` — libera o lock; permite que outro prossiga

Agora `contador` sempre chega a **2000**. O mutex garante que apenas uma goroutine executa `contador++` por vez.

SYNC.RWMUTEX

`sync.RWMutex` diferencia **leitores** (múltiplos simultâneos) de **escritores** (exclusivos).

```go
var (
    dados map[string]string
    mu    sync.RWMutex
)

func ler(chave string) string {
    mu.RLock()              // múltiplos leitores simultâneos
    v := dados[chave]
    mu.RUnlock()
    return v
}

func escrever(chave, valor string) {
    mu.Lock()               // escritor exclusivo
    dados[chave] = valor
    mu.Unlock()
}
```

MÚLTIPLAS LEITURAS

ESCRITA EXCLUSIVA

DETECTOR DE CORRIDA

Go inclui um **detector de data races** integrado ao toolchain — ativado com a flag `-race`.

```
$ go run -race programa.go
==================
WARNING: DATA RACE
Read at 0x00c00001a0b8 by goroutine 7:
  main.incrementar()
      /programa.go:10 +0x64

Previous write at 0x00c00001a0b8 by goroutine 6:
  main.incrementar()
      /programa.go:10 +0x7c
==================
```

- `go run -race` — executa com detecção
- `go test -race` — testa com detecção
- `go build -race` — compila com detecção (mais lento)
- Detecta **em tempo de execução** — só encontra races que efetivamente ocorrem

O detector tem custo de performance (2–10× mais lento). Usar em **desenvolvimento e testes**, não em produção.

CHANNELS VS. MUTEXES

CHANNELS

- **Transferir** dados entre goroutines
- **Sinalizar** eventos (done, cancel)
- **Compor** com select
- Quando o dado **muda de dono**

MUTEXES

- **Proteger** estado interno (caches, contadores, maps)
- **Estado local** de um componente
- Quando o dado **não muda de dono**
- Quando a simplicidade do lock é suficiente

Regra prática: use channels para **comunicação entre goroutines** e mutexes para **proteger estado interno**.

MODELO REATIVO E SELECT

Multiplexação e topologias

MODELO REATIVO

Em um processo **reativo**, a goroutine espera por estímulos em múltiplos canais e reage conforme o que chega primeiro.

- **Problema:** uma goroutine precisa ouvir vários canais ao mesmo tempo
- Leitura simples (`<-ch`) bloqueia em **um só canal**
- Se múltiplos canais estão prontos: qual escolher?
- **Não-determinismo:** a escolha deve ser justa — nenhum canal favorecido sistematicamente

A instrução `select` resolve exatamente esse problema: multiplexação não-determinística de canais.

SELECT EM GO

> `select` espera por múltiplas operações em channels e executa a primeira que estiver pronta.

```go
select {
case msg := <-ch1:
    fmt.Println("De ch1:", msg)
case msg := <-ch2:
    fmt.Println("De ch2:", msg)
case ch3 <- valor:
    fmt.Println("Enviado para ch3")
}
```

- Se **nenhum** case está pronto → **bloqueia** até algum estar
- Se **um** case está pronto → executa esse case
- Se **vários** cases estão prontos → escolhe um **aleatoriamente**
- Cases podem ser de **envio** ou **recepção**

SELECT COM DEFAULT

O `default` transforma o select em **não-bloqueante**: se nenhum canal está pronto, executa imediatamente.

```
select {
case msg := <-ch:
    fmt.Println("Recebido:", msg)
default:
    fmt.Println("Nenhuma mensagem disponível")
    // continua fazendo outras coisas
}
```

QUANDO USAR DEFAULT?

- Para **não bloquear** o programa
- Para fazer **polling** periódico
- Para **evitar deadlocks**

CUIDADO!

- Em loop, gera **busy waiting** (CPU 100%)
- Normalmente combinado com `time.Sleep`
- Prefer `time.After` para timeouts

SELECT COM TIMEOUTS

`time.After` retorna um canal que dispara após o tempo especificado — combinado com `select`, implementa **timeouts**.

TIMEOUT SIMPLES

```
select {
case res := <-ch:
    fmt.Println("OK:", res)
case <-time.After(3 * time.Second):
    fmt.Println("Timeout!")
}
```

LOOP REATIVO COM TIMEOUT

```
for {
    select {
    case msg := <-mensagens:
        processar(msg)
    case <-time.After(30 * time.Second):
        fmt.Println("Encerrando")
        return
    }
}
```

O timeout evita **bloqueio indefinido** — uma das preocupações centrais da troca de mensagens.

RESUMO

- **Canais:** comunicação + sincronização — síncronos (rendez-vous) e assíncronos (bufferizados)
- **Channels em Go:** `make`, envio/recepção, direcionais, `close` / `range`
- **Mutex:** proteção de estado compartilhado — `sync.Mutex` e `sync.RWMutex`
- **Select:** multiplexação não-determinística — `default` e `time.After`

Próxima aula: laboratório de programação concorrente com controle de acesso.